

Module 10: Identity, Access, and Network Hardening

Intro to Networks

Brendan Shea, PhD

Rochester Community and Technical College



Outline I

- 1 Authentication
- 2 Authorization and Account Management
- 3 Network Hardening
- 4 Switch Security
- 5 Network Security Rules

Key Module 9 Ideas

- The **CIA triad**: Confidentiality (keep secrets), Integrity (prevent tampering), Availability (keep systems running).
- Threats include malware, phishing, spoofing, and denial-of-service attacks.
- Defense in depth layers multiple controls so no single failure causes total compromise.

Where Module 10 Takes This

Module 9 described the *threats*. Module 10 gives us the *tools to fight back*: controlling who gets in, what they are allowed to do, and how we lock systems down.

Learning Outcomes I

After completing this module, you will be able to:

- Define **AAA** (Authentication, Authorization, and Accounting) and explain why all three parts matter.
- Compare authentication methods: passwords, MFA, tokens, certificates, and biometrics.
- Explain Single Sign-On (SSO), Kerberos tickets, and federated identity using SAML.
- Describe digital certificates and the chain of trust in a Public Key Infrastructure (PKI).
- Compare RADIUS and TACACS+: when and why each protocol is used.
- Apply authorization models including Role-Based Access Control (RBAC) and the principle of least privilege.
- Describe directory services (LDAP) and explain why encrypting directory traffic (LDAPS) matters.
- Identify network hardening practices: changing defaults, disabling unused services, and patching.

Learning Outcomes II

- Explain switch security controls: port security, 802.1X, DHCP snooping, and BPDU guard.
- Read and troubleshoot Access Control Lists (ACLs), proxy configurations, and content filters.

What is AAA? Your Digital ID Check

AAA stands for **Authentication**, **Authorization**, and **Accounting**. These three steps happen every time someone logs into a network system.

Authentication

Who are you?

Prove your identity. Like showing your ID at the door.

Authorization

What can you do?

Check your permissions. Like a VIP wristband vs. general admission.

Accounting

What did you do?

Keep a record. Like a hotel logging every time a room key is used.

Real-World Connection: Your Bank App

1) You enter your username and password — **Authentication**. 2) The app shows *your* accounts, not someone else's — **Authorization**. 3) Every transaction and login attempt is logged for fraud review — **Accounting**.

Access Control: The Bouncer, the List, and the Log

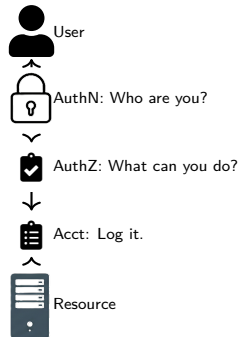
Think of a concert venue:

- The **bouncer** checks your ID → Authentication.
- Your **wristband color** determines which areas you enter → Authorization.
- The **entrance log** records when you came and went → Accounting.

Zero-trust principle: Do not automatically trust users or devices just because they are already connected to the network. Every request should go through the full AAA check.

The Full Login Flow

1) Claim identity → 2) Prove identity → 3) Apply permissions → 4) Log the action.



You Already Do Authentication Every Day

Authentication is not an abstract networking concept — you rely on it constantly in daily life.

Everyday Situation	How You Prove Identity	Network Equivalent
Unlock your phone	Fingerprint or face scan	Biometric authentication
Log into email	Password	Knowledge factor
Bank texts you a code	One-time code via SMS	SMS one-time password (OTP)
Badge into your office	Physical access card	Smart card / hardware token
Show ID at a bar	Government-issued document	Digital certificate

The Pattern

In every case, you prove your identity before being granted access. Network authentication follows the same logic — just at much higher speed and scale.

Authentication Methods: The Three Factors

Every authentication method falls into one of three **factor categories**. Using two or more factors from *different* categories is called **Multi-Factor Authentication (MFA)**.

Something You Know

Passwords, PINs, security questions.

Easy to deploy. Also the easiest for attackers to steal.

Something You Have

Phone app, hardware security key, smart card.

Harder to steal remotely — the attacker needs the physical device.

Something You Are

Fingerprint, face scan, voice pattern.

Very hard to fake, but requires hardware support.

Hansel and Gretel's Workshop

Hansel logs into the workshop system with his password (something he *knows*). The system also requires a code from the authenticator app on his phone (something he *has*). Even if a witch steals his password, she cannot log in without his phone. That is MFA in action.

Why Passwords Alone Are Not Enough

Passwords are the most common authentication method — and the most commonly stolen.

How passwords get stolen (even strong ones)

- **Phishing:** You type your real password into a fake login page that looks identical to the real one.
- **Data breach:** A site you used was hacked. Attackers try that same password on your bank, email, and work accounts.
- **Credential stuffing:** Automated tools try millions of stolen username/password combos against popular sites every day.

MFA as a second lock

Imagine your front door has two independent locks: a deadbolt and a PIN keypad. A thief who copies your key still cannot get in without the PIN. MFA works the same way — a stolen password alone is not enough to log in.

Beginner rule

The more important the account, the stronger the second factor should be. Admin accounts, banking, and email deserve your best MFA.

Phishing-Resistant MFA: Not All Second Factors Are Equal

Not all MFA protects equally. Some second factors can still be bypassed by a sophisticated fake login page.

Comparing MFA strength — weakest to strongest

- **SMS text code:** Better than a password alone, but an attacker running a fake site in real time can relay the code before it expires.
- **Authenticator app (TOTP — Time-based One-Time Password):** Good for most users. 30-second codes limit the window, but a fast attacker can still relay them.
- **Hardware security key or Passkey:** The strongest option. The key cryptographically verifies the *real domain name* of the site. A fake site gets a response that is useless to the attacker.

Practical guidance

For everyday personal accounts: use an authenticator app. For admin accounts and sensitive systems: deploy hardware keys or passkeys. The extra friction is worth it.

Local Authentication

Local authentication means each device stores and manages its own user accounts and passwords directly — no central server is involved.

Where it is useful

- Emergency "break-glass" access when central identity servers are unreachable.
- Small isolated labs or edge devices with limited features.
- Initial setup before a central identity system is configured.

Why it is risky at scale

- Each device has its own passwords; they drift out of sync over time.
- When someone leaves the organization, you must manually remove their account from every device — and it is easy to miss one.
- Audit logs are fragmented across dozens of devices and nearly impossible to correlate.

Design rule

Keep one documented emergency account per critical device, tested quarterly. Use centralized authentication for all normal logins.

What is Single Sign-On (SSO)?

Single Sign-On (SSO) means you authenticate *once* and can then access many different services without entering your credentials again.

The Theme Park Wristband Analogy

At a theme park, you pay and register at the front gate once. The wristband you receive lets you onto every ride and into every restaurant without repeating the check-in process. SSO is that wristband for your network and cloud services.

Without SSO

Separate login for email. Separate login for HR.
Separate login for file server. Separate login for project tracker. Four passwords, four login pages, four places where a stolen credential causes damage.

With SSO

Log in once when you start your day. Email, HR, file server, and project tools all open without additional prompts. One strong credential to protect, one place to revoke when someone leaves.

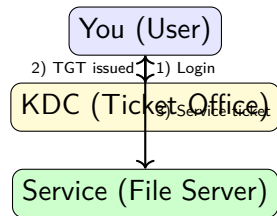
How SSO Works: Kerberos and Tickets

Kerberos is the protocol that powers SSO in most corporate Windows environments (Active Directory). Instead of sending your password to every service, you get a temporary **ticket** from a trusted authority and present that ticket to access services.

Three actors in Kerberos

- **You:** the user requesting access.
- **KDC (Key Distribution Center):** the trusted ticket-issuing authority.
- **Service:** the resource you want to reach (file server, printer, app).

TGT (Ticket Granting Ticket): your master pass, issued at login, exchanged for individual service tickets without re-entering your password.



Your password never travels to the file server. Only a short-lived ticket does. Less exposure means less risk.

Digital Certificates: Your Network ID Card

A **digital certificate** is an electronic document that proves a server (or user) is who it claims to be.

Think of it like a passport:

- It contains identity information (name, domain, public key).
- A trusted authority — a **Certificate Authority (CA)** — has verified the identity and signed the certificate.
- Others trust it because they trust the issuer.

You encounter certificates constantly:

- The padlock in your browser = HTTPS certificate.
- VPN connections.
- Secure Wi-Fi (802.1X/EAP-TLS).

PKI (Public Key Infrastructure) is the full system of CAs, certificates, and trust rules.

Chain of trust:

- A **Root CA** is the ultimate trusted authority.
- **Intermediate CAs** sit below the Root CA and issue certificates to servers and users.
- Your browser ships with a built-in list of trusted Root CAs.

Beginner checkpoint

If a site's certificate is expired, unsigned, or doesn't match the domain name, your browser shows a warning. That warning means the chain of trust is broken — do not proceed.

Key Management: Protecting the Keys to the Kingdom

Encryption is only as strong as the protection around the **keys** used to lock and unlock data. Having encryption and managing keys poorly is like locking your door and leaving the key under the mat.

Key Lifecycle

- 1 **Generate:** Create keys with strong randomness (entropy).
- 2 **Store:** Keep private keys in secure hardware vaults, not in code or configuration files.
- 3 **Rotate:** Replace keys on a regular schedule, or immediately after a suspected compromise.
- 4 **Revoke/Destroy:** Retire old keys completely — no stale copies.

Common Failures

- Hardcoded credentials or keys in source code (attackers search GitHub for these automatically).
- One private key shared across many servers.
- Long-lived keys that are never rotated.

Revocation: CRL and OCSP

When a certificate is compromised, it is added to a **CRL** (Certificate Revocation List) or checked via **OCSP** (Online Certificate Status Protocol) so systems stop trusting it immediately.

Federated Identity and SAML

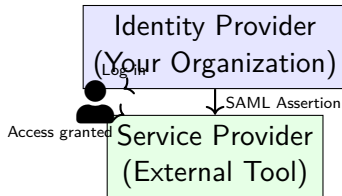
Federated identity lets one organization trust another organization's authentication. You log in with your home organization's credentials and get access to a partner's service — no second account needed.

Real-world examples: “Log in with Google,” “Sign in with your university account” on a third-party tool.

SAML in Plain Language

SAML (Security Assertion Markup Language) is the standard protocol that carries federated login information.

- The **Identity Provider (IdP)**: your home org that knows who you are.
- The **Service Provider (SP)**: the external site you want to use.
- The IdP sends a signed, tamper-proof **assertion** to the SP: “Yes, this is Alice; she works here and has these attributes.”



Why it matters: One account to manage, one place to deprovision when someone leaves.

Remote Authentication: RADIUS and TACACS+

When many users log into network devices or the corporate network from different locations, authentication needs to happen on a *central server* — not device-by-device. Two protocols handle this:

RADIUS

Remote Authentication Dial-In User Service

- Common use: Wi-Fi login (802.1X), VPN, wired port authentication.
- Combines authentication and authorization in a single response.
- Uses UDP (connectionless, faster but less reliable transport).

Think: A fast ID check at the building entrance — approve or deny, quickly.

TACACS+

Terminal Access Controller Access-Control System Plus

- Common use: admin logins to routers, switches, and firewalls.
- Fully separates Authentication, Authorization, and Accounting.
- Uses TCP (reliable) and encrypts the entire payload.
- Can control which *specific commands* each user is allowed to run.

Think: A detailed background check with a full activity log.

Case Study: Cinderella's Co-Working Login Chaos

Scenario

Cinderella manages a small startup from a co-working space. Her team uses three separate cloud (SaaS) tools: a CRM for client contacts, a source-code repository, and an invoicing app. Each tool has its own independent login system. A contractor's last day is Friday at 5 PM. Cinderella manually removes their account from the CRM. On Monday morning, finance discovers unauthorized invoice edits and a new vendor payout rule that nobody approved. Investigation reveals:

- The contractor's account was removed from the CRM but remained active in the invoicing app.
- Login records show access from a home IP address at 11:42 PM Sunday night.

Discussion Questions

- 1 Which identity management flaw caused this? (What happens when offboarding is manual and spread across separate systems?)
- 2 What single architectural change would have prevented the Sunday-night access?
- 3 What logs would you need to determine the full scope of the damage?

Case Study Solution: Cinderella's Co-Working Login Chaos

Recommended Response

- ❶ **Root cause:** Separate identity stores with no central management. Manual offboarding missed one system. This is exactly the problem centralized identity (SSO) solves — remove access in one place and it is gone everywhere.
- ❷ **Structural fix:** Centralized identity with automated deprovisioning. When the contractor is removed from the identity provider, all three tools lose access simultaneously. MFA would have made the Sunday-night login much harder even with valid credentials.
- ❸ **Containment:** Revoke all active sessions immediately, freeze invoice workflow changes, and revert unauthorized records.
- ❹ **Audit data:** Correlate login timestamps, session IDs, source IP addresses, and change records across all three tools to establish the complete impact timeline.

Key Principle

Identity controls are only as strong as your offboarding process. If offboarding is manual, it will eventually be incomplete. Automate it.

From Authentication to Authorization

Authentication answered the question: *“Who are you?”*

Authorization answers the next question: *“Now that we know who you are — what are you allowed to do?”*

A Hospital Analogy

A doctor and a janitor both badge in at the hospital entrance. Both **authenticated** successfully — the system knows who they are. But the doctor can open patient records; the janitor cannot. The difference in what each person can *do* is **authorization**.

The most common access mistake

Most security incidents involving insider access are not sophisticated attacks. They are *permission mistakes*: someone had access they should never have had, and eventually it was abused — or stolen by an outsider. Getting authorization right matters just as much as getting authentication right.

Authorization and Role-Based Access Control (RBAC)

RBAC (Role-Based Access Control) is the most common authorization model. Instead of assigning permissions to each individual person, you define **roles** and grant permissions to those roles.

How RBAC Works

- 1 Define job roles (Helpdesk, Network Admin, Security Analyst, Intern).
- 2 Map the *minimum* permissions each role actually needs.
- 3 Assign users to roles, not to individual permissions.
- 4 When someone changes jobs, change their role — not dozens of individual permissions.

Example Role Mapping

- **Helpdesk:** Reset passwords, create support tickets.
- **Network Admin:** Configure switches and routers.
- **Security Analyst:** Read logs only, no configuration rights.
- **Intern:** Read-only access to training materials.

Least Privilege

Least privilege means giving each user the minimum access needed to do their job — and nothing more. Excess permissions are a hidden, silent risk sitting in your system waiting to be exploited.

Privileged Access Management (PAM)

PAM (Privileged Access Management) provides extra controls specifically for **admin-level accounts** — the accounts that can change configurations, access everything, and cause the most damage if compromised.

Core PAM controls

- **Vault credentials:** Store admin passwords in an encrypted vault, not in shared spreadsheets or sticky notes.
- **Just-in-time (JIT) elevation:** Grant admin rights only for a specific approved task, then remove them automatically.
- **Approval workflows:** High-risk actions require a supervisor to approve first.
- **Session recording:** Every privileged session is recorded for investigation and compliance.

Quick wins even without a PAM tool

- Eliminate shared admin accounts — every person needs their own named account.
- Never use your admin account for everyday tasks like email or web browsing.
- Log all privileged actions to a central, tamper-resistant syslog server.

What “good” looks like

Admin rights are temporary, approved, recorded, and automatically expired. Nobody uses root or Administrator for daily work.

LDAP: The Network's Phone Book

LDAP (Lightweight Directory Access Protocol) is the standard protocol for looking up user accounts, group memberships, and identity attributes from a central directory — like Microsoft Active Directory.

Think of it as the network's **phone book**: when an application needs to know “Is Alice in the Engineering group?” it asks the directory server using LDAP, and the directory answers.

- Single, authoritative source for who exists and what groups they belong to.
- Applications use LDAP group membership to make authorization decisions.
- Supported across Linux, Windows, network devices, and cloud services.

Reading an LDAP Address (DN)

Every entry in the directory has a unique address called a **Distinguished Name (DN)**:

`cn=alice,ou=engineering,dc=example,dc=edu`

Reading right to left: the domain is `example.edu`, the organizational unit is `engineering`, and the user's common name (`cn`) is `alice`.

Operational note

If group memberships are wrong in LDAP, authorization will be wrong in every application that reads from it.

LDAPS: Encrypting the Phone Book

Plain **LDAP** (port 389) sends directory queries and credentials across the network in *cleartext*. Anyone who can capture the traffic can read usernames, passwords, and group data.

LDAPS (LDAP Secure, port 636) wraps LDAP traffic in **TLS** encryption — the same technology that makes HTTPS secure. Credentials and query results are protected in transit.

StartTLS is an alternative approach: the connection starts as plain LDAP on port 389, then upgrades to encrypted mid-session before any credentials are sent.

LDAPS Hardening Checklist

- **Disable anonymous binds:** Do not let unauthenticated queries browse the directory.
- **Enforce TLS 1.2 or higher:** Older TLS versions have known weaknesses.
- **Restrict bind accounts:** The service account apps use to query LDAP should be read-only with minimal permissions.
- **Log failed bind attempts:** Repeated failures may indicate a brute-force attack against directory credentials.

Example: Verifying a Secure LDAP Connection

On Linux, the `ldapsearch` command-line tool can test whether an LDAPS connection is working. A successful result confirms the encrypted connection is established and the server certificate is trusted.

Connecting securely on port 636 (LDAPS)

```
$ ldapsearch -H ldaps://ad.example.com:636 \  
            -D "cn=admin,dc=example,dc=com" -W  
Enter LDAP Password:  
# extended LDIF  
# LDAPv3  
# LDAP successful bind ...
```

If the connection fails, check in order

- 1) Is the server's TLS certificate valid and trusted by this client?
- 2) Is port 636 open through the firewall?
- 3) Is the directory server configured to accept LDAPS connections?

Case Study: Little Red's Hospital Access Mix-Up

Scenario

Little Red works at a hospital help desk. During her first week, she notices something alarming: interns starting their rotation can open both help desk ticketing tools *and* the confidential payroll and HR compensation records. After checking the directory, she finds one broad LDAP group called `staff_all` that grants access to everything simultaneously.

Additional details:

- Two temporary interns were provisioned using the full-time employee account template, giving them far more access than their role requires.
- A shared account named “HRAdmin” was used during weekend support shifts — no one knows exactly who used it or when.
- There is no approval record or audit trail for payroll export actions taken over the past six months.

Discussion Questions

- 1 Which principles failed: least privilege, separation of duties, or both? Explain.
- 2 How should the LDAP group structure be redesigned?
- 3 What PAM controls should be added for payroll and HR admin functions?

Case Study Solution: Little Red's Hospital Access Mix-Up

Recommended Response

- ❶ **Both principles failed.** Interns had access far beyond their job scope (*least privilege* violation). The same account could perform both help desk and payroll functions (*separation of duties* violation). The shared HRAdmin account makes individual accountability impossible.
- ❷ **Directory redesign:** Replace `staff_all` with purpose-specific groups: `intern_readonly`, `helpdesk`, `hr_analyst`, `payroll_admin`. Each group maps to only the minimum permissions that role actually requires.
- ❸ **PAM controls:** Eliminate the shared HRAdmin account; require individual named accounts for all HR work. Add just-in-time elevation with manager approval for payroll exports. Record all privileged HR sessions.
- ❹ **Governance:** Implement quarterly access reviews where managers certify which users still need each sensitive permission.

What Does “Hardening” Actually Mean?

Hardening means configuring a device or system to remove unnecessary features, close unused access points, and replace insecure defaults that attackers already know about.

The New Router Analogy

A brand-new home router ships from the factory with:

- Username: `admin` Password: `admin`
- Remote management enabled from the internet
- Firmware with known, publicly documented vulnerabilities

Millions of routers ship with *identical* default credentials. Attackers automate internet-wide scans for them within days of a new model's release.

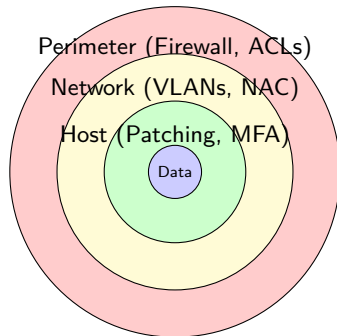
Hardening that router: change the admin password, disable internet-facing remote management, and update the firmware. You go from “default target” to “not worth the effort.”

Rule of thumb

If a feature or service is not actively being used, disable it. Every unused service is an open door you may have forgotten.

Defense in Depth: No Single Control Is Enough

Defense in depth means building multiple independent layers of security so that bypassing one control does not result in complete compromise.



An attacker who bypasses the firewall still faces VLANs, host hardening, and encryption around the data itself.

Common beginner mistake: Trusting only the firewall and neglecting identity controls, patching, and monitoring.

Layered Hardening: A Small Office Example

Defense in depth does not require expensive tools. A small office can layer simple, affordable controls effectively.

Layer	Concrete Control	What It Prevents
Identity	MFA for all remote logins	Stolen passwords used to log in remotely
Network	VLANs separating guest from staff	Guest devices reaching internal file servers
Host	Automatic patching + local firewall	Known exploits and exposed local services
Physical	Lock server closet, disable unused ports	Someone plugging in a rogue device
Monitoring	Central syslog and email alerts	Attacks that slip through other layers

Key insight

When one layer fails, the remaining layers slow the attacker down, buy time for detection, and create an audit trail that supports incident response.

Device and Service Hardening: The Checklist

Device hardening steps

- ➊ Change all default credentials immediately on first login.
- ➋ Disable interfaces, ports, and management services that are not in use.
- ➌ Restrict management access: only allow SSH connections from specific trusted admin IP addresses.
- ➍ Apply firmware and security patches on a regular, documented schedule.
- ➎ Use encrypted management protocols only: SSH instead of Telnet, HTTPS instead of HTTP.

Service hardening

- Remove or disable any service not actively needed (every running process is a potential target).
- Replace insecure protocols with secure equivalents:
 - Telnet → SSH
 - FTP → SFTP
 - HTTP → HTTPS
 - SNMPv1/v2c → SNMPv3
- Enable host-based firewalls on servers.
- Send all service logs to a central, tamper-resistant syslog server.

Network Access Control (NAC) and Port Security

Network Access Control (NAC) means the network itself verifies whether a device is permitted before granting full access.

Think of an airport: you do not walk straight to the gate. You clear a security checkpoint first. NAC is that checkpoint for your network ports.

Port security is a simpler, switch-level form of this: limit which devices — identified by their **MAC address** (the hardware ID burned into every network interface) — can connect to each port.

- Set a maximum of 1–2 MAC addresses per access port.
- **Sticky learning**: the switch remembers the first device it sees.
- Violation actions: **protect** (drop silently), **restrict** (drop and log), or **shutdown** (disable the port entirely).

Why this matters

- Prevents someone from plugging an unauthorized device into a wall jack.
- Stops a visitor from adding an unmanaged hub or mini-switch to multiply access.

Basic design pattern:

- 1 Authenticate the device or user.
- 2 Assign to the correct VLAN based on identity.
- 3 Monitor port activity continuously.

Design tip

Build a secure template for access ports. Every new port is locked down by default, not open by default.

Example: Port Security on a Lobby Switch

This Cisco switch configuration restricts a lobby port to exactly one device. The switch learns and remembers the first MAC address it sees ("sticky"), and shuts the port if anything else connects.

Cisco IOS port security configuration

```
interface GigabitEthernet1/0/2
  switchport mode access
  switchport port-security
  switchport port-security maximum 1
  switchport port-security mac-address sticky
  switchport port-security violation shutdown
```

What happens when a second device connects?

```
%PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred,
port Gi1/0/2 is shutting down.
```

The port disables itself and requires manual administrator recovery. One wall jack, one expected device, one known MAC address.

What is IEEE 802.1X? The Velvet Rope at the Network Door

802.1X is a standard that controls access to a network port or Wi-Fi connection at the *identity* level — not just the device level.

Analogy: The Nightclub Velvet Rope

A bouncer does not let anyone through just because they walked up. They check your ID and verify you are on the list. With 802.1X, your laptop may be physically plugged into a switch port — but *no network traffic flows* until you have successfully authenticated. The port is the velvet rope.

Supplicant

Your laptop or phone — the device *requesting* network access.

Authenticator

The switch or Wi-Fi access point that *enforces* the check. It holds traffic until authentication succeeds.

Auth Server

Usually a RADIUS server that *validates* the credentials and tells the switch: permit or deny.

EAP (Extensible Authentication Protocol) is the “conversation format” used between these three parties. Common methods: **EAP-TLS** (certificate-based, strongest phishing resistance) and **PEAP** (username/password inside an encrypted tunnel).

Port Guards: Stopping Layer 2 Attacks

Layer 2 (the switch layer) has specific vulnerabilities. Guard features are built into managed switches to block them.

Guard Features Explained

- **BPDU Guard:** BPDUs (Bridge Protocol Data Units) are control messages switches exchange to manage spanning tree topology. Only switches should send them. BPDU Guard shuts a port the moment BPDUs arrive from a device that should not be sending them — like a rogue switch someone plugged in.
- **Root Guard:** Prevents any device from claiming to be the “root bridge” (the master reference switch), which would disrupt the entire switching topology.
- **DHCP Snooping:** Blocks unauthorized DHCP servers from handing out fake IP addresses and gateway settings.

Why This Matters

Without these guards, a single rogue device plugged into a wall jack can:

- Intercept and read all traffic on the segment (man-in-the-middle).
- Hand out fake IP settings, making the network unreachable.
- Disrupt spanning tree and bring down switching for the entire floor.

Apply these guards to all access ports. Explicitly exclude trunk and uplink ports (which legitimately carry BPDUs and multiple VLANs).

Example: Recovering from a Port Violation

When BPDU Guard or port security triggers, the switch places the port in err-disabled state. Nothing can use the port until an administrator manually recovers it — which is intentional.

What you see on the console

```
%PM-4-ERR_DISABLE: psecure-mac-violation error detected on Gi1/0/2,  
putting Gi1/0/2 in err-disable state
```

```
Switch# show interfaces status err-disabled
```

Port	Name	Status	Reason
Gi1/0/2	Lobby_Printer	err-disabled	psecure-mac-violation

Recovery steps

- 1 Physically remove the unauthorized device from the port.
- 2 Enter interface configuration: `interface GigabitEthernet1/0/2`
- 3 Run `shutdown` then `no shutdown` to reset the port state.
- 4 Confirm recovery: `show interfaces GigabitEthernet1/0/2 status`

Port Mirroring (SPAN): Watching Without Touching

Port mirroring (also called **SPAN** — Switched Port ANalyzer) copies traffic from one or more switch ports to a dedicated monitor port. A security or analysis tool connected to that monitor port can see all the traffic without being in the path of that traffic.

Use Cases

- Security investigation: capture and replay suspicious traffic for analysis.
- **IDS** (Intrusion Detection System) placement: the IDS watches all traffic and raises alerts without slowing down real traffic.
- Performance troubleshooting: a packet analyzer watches live traffic to diagnose slowdowns.

Caution

Mirrored traffic contains *everything* — passwords, personal data, confidential files, health records. Treat the monitor port and capture device as high-security assets.

Best practice: Mirror only the specific ports or VLANs relevant to your investigation. Mirroring everything creates excessive data and can overwhelm both analysis tools and storage.

Case Study: Jack and the Beanstalk Data Center

Scenario

Jack's cloud startup rents space on a shared data center floor with contractor access during business hours. During a maintenance window, a visiting contractor plugs a small unmanaged switch (a "mini-switch") into an open wall port to get extra network ports for their equipment — without asking anyone.

Within the next hour:

- Nearby servers begin receiving incorrect default gateway addresses.
- VoIP phones start dropping calls and flapping between VLANs.
- Network monitoring shows unexpected BPDU traffic and multiple unauthorized DHCP offer packets originating from an access port.

Discussion Questions

- 1 Which switch security controls would have blocked this before it started?
- 2 Which guard features should be enabled on all access ports in this environment, and why?
- 3 How could port mirroring help confirm exactly what the mini-switch was doing?

Case Study Solution: Jack and the Beanstalk Data Center

Recommended Response

- ➊ **Prevention:** 802.1X authentication would have blocked the unauthenticated mini-switch from gaining any network access at all. Port security with a MAC limit of 1 would have shut the port the moment a second device appeared behind it.
- ➋ **Guard features:** DHCP Snooping blocks the rogue DHCP server on the mini-switch. BPDU Guard shuts the port the moment spanning-tree BPDUs arrive from a non-switch device. DAI prevents any ARP spoofing the mini-switch might attempt to intercept traffic.
- ➌ **SPAN:** Mirror the affected access port to a capture device. Analyze the PCAP to confirm rogue DHCP offers, BPDU floods, and any ARP manipulation — then document timestamps and affected endpoints for the incident report.
- ➍ **Operational fix:** Disable and label all unused ports. Contractor access to specific authorized ports only, documented in the change management system.

Access Control Lists (ACLs): Traffic Rules for the Network

An **ACL** (Access Control List) is an ordered list of rules on a router or firewall that decides whether to **permit** or **deny** each packet.

Three rules to understand first:

- Rules are checked **top to bottom**. The **first matching rule wins** — checking stops immediately.
- At the end of every ACL there is an **implicit deny all**. If no rule matches, the packet is dropped silently.
- **Standard ACLs** filter by source IP address only. **Extended ACLs** can filter by source, destination, protocol, and port number.

Think of it as a checklist at a border crossing:

- 1 Where did this packet come from?
- 2 Where is it going?
- 3 What protocol and port is it using?
- 4 Does the first matching rule say permit or deny?
- 5 No match = implicit deny. Packet dropped.

Most common beginner mistake

A broad `permit ip any any` rule near the top of the list quietly cancels every specific deny rule below it — because the broad rule matches everything first.

Example: Reading a Cisco ACL

Read this ACL line by line. It is applied to a guest Wi-Fi segment to control what guests can reach.

Cisco IOS Extended ACL — GUEST_RESTRICTION

```
ip access-list extended GUEST_RESTRICTION
 10 permit udp any any eq 53      ! Allow DNS (port 53) -- guests need name lookups
 20 permit tcp any any eq 80      ! Allow HTTP (port 80)
 30 permit tcp any any eq 443     ! Allow HTTPS (port 443)
 40 deny ip any 10.0.0.0 0.255.255.255 ! Block internal network (10.x.x.x)
 50 permit ip any any             ! Allow all other internet traffic
```

Why order matters critically

If lines 40 and 50 were swapped, line 50 (`permit ip any any`) would match *every* packet first — including traffic destined for the internal 10.0.0.0 network. Line 40 would never be evaluated. Guests could reach internal servers.

Proxy Servers: The Helpful Middleman

A **proxy server** sits between users and the internet (or between the internet and your servers). All traffic is routed through the proxy, where it can be inspected, filtered, logged, and controlled before reaching its destination.

Why use a proxy?

- Require authentication before any internet access is granted.
- Block malicious or policy-violating websites.
- Cache frequently accessed content (faster for users, cheaper on bandwidth).
- Maintain a complete log of every URL users visit.
- Scan downloads for malware before they reach the endpoint.

Three Proxy Types

- **Forward proxy:** Internal users go *out* through the proxy to the internet. The proxy protects and monitors your users.
- **Reverse proxy:** External visitors come *in* through the proxy to reach your servers. The proxy protects your servers and can load-balance across them.
- **Transparent proxy:** Intercepts traffic automatically with no configuration required on user devices — users may not know it exists.

Forward Proxy vs. Reverse Proxy

Feature	Forward Proxy	Reverse Proxy
Traffic direction	Outbound (users to internet)	Inbound (internet to your servers)
Protects	Internal users	Internal servers
Typical jobs	URL filtering, malware scan, caching	Load balance, WAF, TLS termination
Does client know?	Usually (configured on device)	No (transparent to outside visitors)
Example	Corporate web filter	nginx in front of a web application

Snow White's Studio

Snow White's editors browse the internet through a **forward proxy** that blocks malware-hosting domains. External clients previewing completed videos hit a **reverse proxy** that distributes load across render servers and hides their real addresses.

Content Filtering: More Than Blocking Websites

Content filtering enforces organizational policy about what users can access, what files can be downloaded, and which domains can be contacted.

What Content Filtering Can Do

- **URL category blocking:** Block categories such as gambling, malware-hosting, or social media during work hours.
- **DNS filtering:** Block known malicious domains before a TCP connection is even established.
- **File type control:** Block executable files (.exe, .ps1) downloaded from unknown sources.
- **Malware scanning:** Inspect downloaded files before they reach the user's device.
- **Safe search enforcement:** Force safe-search mode on major search engines.

Balance Security and Usability

Overly aggressive filtering breaks legitimate business tools. A security researcher may need to visit sites that a “hacking tools” category would block. Work with users, not against them.

Deployment best practice

Start in **monitor-only mode**: log what *would* be blocked without actually blocking it. Review logs for two weeks to identify false positives, then switch to enforcement.

Misconfigured Firewalls and ACLs: How Rules Go Wrong

Common Misconfigurations

- **Shadowed rules:** A broad rule earlier in the list matches all the traffic that a specific rule below it was meant to catch. The specific rule is “shadowed” and never runs.
- **Any-any permit in production:** A catch-all allow rule left over from initial testing that was never removed.
- **Wrong rule order:** Deny rules placed below the permit rules they were meant to override.
- **Missing egress filtering:** Outbound traffic from your network is not filtered — data exfiltration has no obstacle.

Most ACL-related outages trace to rule *order* mistakes, not missing rules.

Troubleshooting Approach

- 1 Write down the intended traffic matrix *first*: what should be allowed, from where, to where.
- 2 Trace the packet path and identify every firewall or ACL it crosses.
- 3 Check hit counters — a rule with zero hits is likely shadowed or unreachable.
- 4 Make the minimum change needed and retest before moving to the next change.

Always have a tested rollback plan ready before applying changes in production.

Case Study: Snow White's Streaming Studio Firewall Meltdown

Scenario

Snow White runs a media startup with seven remote editors and an on-premises render cluster. After a weekend firewall change, editors cannot sync project files or push final exports to the render cluster. General web browsing and social media still work fine.

A review of the firewall ruleset reveals:

- A broad `permit ip any any` rule is positioned near the top of the list.
- A new deny rule targeting the file-sync service's ports is placed *below* a general internet-allow rule.
- The change ticket has no documentation of which business traffic flows the change was intended to protect or affect.

Discussion Questions

- 1 What ACL or firewall ordering problem is most likely responsible for the editors' issues?
- 2 Which traffic flows should be explicitly permitted first in a corrected rule set?
- 3 Where should content filtering be introduced to reduce risk without breaking editor workflows?

Case Study Solution: Snow White's Streaming Studio Firewall Meltdown

Recommended Response

- ❶ **Shadowed deny rule.** The `permit ip any any` rule above the file-sync deny matches all packets first, so the deny is never reached. The sync service traffic is being permitted by the broad rule, but something else in the path (possibly a different ACL or routing issue) is blocking it. Audit every policy enforcement point in the editor-to-render path.
- ❷ **Rule rebuild from a traffic matrix:** Explicitly permit editor-to-render traffic (specific source subnets, destination IPs, required ports). Permit sync service traffic by protocol and port. Place all specific permits *before* any broad denies. Remove the any-any permit.
- ❸ **Content filtering:** Route general internet browsing through an authenticated forward proxy with URL category filtering. Keep render cluster and sync traffic on a separate, unfiltered path. Document the distinction in the change ticket.

Process lesson

No firewall change should go into production without a traffic matrix and a rollback plan documented in the change ticket.

Module 10 Summary I

- **AAA** (Authentication, Authorization, Accounting) is the framework behind every secure login: prove identity, apply permissions, log all activity.
- **MFA** (Multi-Factor Authentication) combines independent factors to make stolen passwords insufficient on their own. Hardware keys and passkeys provide the strongest phishing resistance.
- **SSO** and **Kerberos** let users authenticate once and access many services via temporary tickets. **SAML** extends this trust across organizational boundaries (federated identity).
- **PKI** and digital certificates establish a chain of trust for encrypted communications. Key lifecycle management — generate, store, rotate, revoke — is as important as the encryption itself.
- **RADIUS** handles endpoint and user access (Wi-Fi, VPN). **TACACS+** handles network device administration with full command-level control and separation of AAA functions.
- **RBAC** and least privilege minimize the blast radius of any compromised account. **PAM** adds vaulting, just-in-time elevation, and session recording specifically for high-risk admin credentials.

Module 10 Summary II

- **LDAP** centralizes identity lookups. **LDAPS** encrypts that traffic in transit and should replace plain LDAP in all production environments.
- **Hardening** removes default credentials, disables unused services, and closes unnecessary access paths before attackers can exploit them.
- Switch controls — 802.1X, port security, DHCP Snooping, BPDU Guard, and DAI — defend the access layer from both physical and Layer 2 attacks. Port mirroring (SPAN) enables visibility without inline risk.
- **ACLs** enforce traffic policy on routers and firewalls; rule order is critical and an implicit deny applies to all unmatched traffic. Proxies and content filters extend policy enforcement to the application layer.

Identity + Hardening Complete



Any questions?